

# Glitch Hunter — Tech Stack

Sep 29, 2026 · @Tanmay Singh

## What this is

Glitch Hunter is a PPO reinforcement-learning agent that plays a Super Mario Bros clone and is watched live through a web dashboard; a second pipeline turns anything it catches breaking the game's own rules into an evidence bundle (screenshot, GIF, Markdown/PDF report). The table below lists every technology in the project and exactly which part of it uses that technology.

## Stack by objective

Each objective built on the one before it: nothing was replaced, only added.

| Objective                               | What it produced                                                                                                                                           | Technologies it added                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1 — Autonomous exploration agent</b> | the 6M brain (46.8% level completion)                                                                                                                      | Python 3.12, Pygame, Gymnasium, PyTorch, Stable-Baselines3 PPO (CnnPolicy), NumPy, cloudpickle, Flask + Flask-SocketIO dashboard, OpenCV frame stream, HTML/CSS/JS, pytest, Git/GitHub                                                                                                                                                                                        |
| <b>2 — QA / glitch-hunting agent</b>    | the 16M QA brain (84.25% coverage, 56.6% completion)                                                                                                       | shared-memory coverage bitmap (NumPy + multiprocessing), reachability mask (.npz), QA reward and lifecycle, anchor-KL consolidation callback, 500-episode evaluation (multiprocessing Pool), SHA-256 artifact manifest, ruff, mypy, pytest-cov, GitHub Actions, uv                                                                                                            |
| <b>3 — Automated bug reporting</b>      | incidents with evidence, GIF/MD/PDF reports, replay; 6 benchmark bugs, all detected; a clean run that reaches the castle gets a "No Bugs Found" run report | incident pipeline and store, Pillow (GIF), fpdf2 (PDF), subprocess replay, zip bundles, REST API + Socket.IO events (incl. live status), a four-tab dashboard (Overview, Live Testing, How It Works, Bug History), a project-facts reader (dashboard_facts.py), game variants (mario_clean / mario_bugged), collision and jump-physics detectors, level-design reference file |

## Language, runtime and environment

| Technology | Version        | What it is               | Where it is used                                                               | Objectives |
|------------|----------------|--------------------------|--------------------------------------------------------------------------------|------------|
| Python     | 3.12 (exactly) | the programming language | the whole codebase: game wrapper, training, dashboard, reporting, tools, tests | 1, 2, 3    |

| Technology                          | Version                           | What it is                  | Where it is used                                                                                                                                                                                                                                                                                                                                                                                                                                             | Objectives |
|-------------------------------------|-----------------------------------|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| venv ( <code>venv_gpu/</code> )     | —                                 | isolated Python environment | holds every dependency; <code>run_dashboard.bat</code> and all commands use <code>venv_gpu\Scripts\python.exe</code>                                                                                                                                                                                                                                                                                                                                         | 1, 2, 3    |
| pip + <code>requirements.txt</code> | —                                 | package installer           | the 2-step manual install (torch first, then <code>requirements.txt</code> )                                                                                                                                                                                                                                                                                                                                                                                 | 1, 2, 3    |
| uv + <code>uv.lock</code>           | —                                 | fast, locked installer      | one-command install ( <code>uv sync</code> ) that encodes the torch/CUDA rules                                                                                                                                                                                                                                                                                                                                                                               | 2, 3       |
| NVIDIA CUDA 12.6                    | cu126 build                       | GPU computing               | fast PPO training on the GPU (CPU-only also works, just slower)                                                                                                                                                                                                                                                                                                                                                                                              | 1, 2       |
| Windows / Linux / macOS             | Windows 11; Ubuntu and macOS (CI) | operating systems           | development and dashboard on Windows; CI tests on all three                                                                                                                                                                                                                                                                                                                                                                                                  | 1, 2, 3    |
| Python standard library             | 3.12                              | built-in modules            | <code>multiprocessing</code> (training workers, shared memory, parallel evaluation), <code>threading/queue</code> (dashboard game thread, report worker), <code>subprocess</code> (incident replay, git, the dashboard's background test count), <code>hashlib</code> (SHA-256), <code>json</code> , <code>zipfile</code> , <code>difflib</code> (bug-diff pin), <code>argparse</code> (tools), <code>ctypes</code> (Windows window control, power settings) | 1, 2, 3    |

## Game engine and simulation

| Technology                                      | Version                     | What it is                              | Where it is used                                                                                                                                 | Objectives |
|-------------------------------------------------|-----------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Pygame                                          | 2.6.1                       | 2D game library (on SDL 2.28)           | runs the Super Mario Bros clone: drawing, sprites, collisions, the game window                                                                   | 1, 2, 3    |
| Mario clone ( <code>games/mario_clean/</code> ) | vendored, pinned by SHA-256 | the game itself (Level 1-1)             | the untouched baseline every brain is trained and measured on                                                                                    | 1, 2, 3    |
| <code>games/mario_bugged/</code>                | copy of the clone           | the game with 6 declared benchmark bugs | bug-detection testing; chosen on the dashboard's Live Testing tab                                                                                | 3          |
| Gymnasium                                       | 1.3.0                       | standard RL environment API             | <code>custom_mario_env.py</code> turns the game into an environment: 10 discrete actions, <code>step()</code> / <code>reset()</code> , info dict | 1, 2, 3    |

| Technology                             | Version | What it is            | Where it is used                                                                                                                                                                                                                             | Objectives |
|----------------------------------------|---------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Gymnasium wrappers                     | 1.3.0   | observation pipeline  | frame skip 4, grayscale, resize to 84×84, stack of 4 frames ( <code>wrap_observation</code> ); <code>TimeLimit</code> in evaluation                                                                                                          | 1, 2, 3    |
| SDL dummy video driver                 | —       | headless rendering    | training workers, evaluation, tests and incident replay run with no window                                                                                                                                                                   | 1, 2, 3    |
| Windows <code>user32</code> via ctypes | —       | native window control | <code>game_window.py</code> : centre, show, hide and minimise the dashboard's game window to the taskbar; <code>desktop.py</code> : centre the terminal and browser windows, and keep full speed on battery (power mode, process throttling) | 1, 3       |

## Reinforcement learning and machine learning

| Technology                  | Version         | What it is                           | Where it is used                                                                                                                                                                                           | Objectives |
|-----------------------------|-----------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| PyTorch                     | 2.14.0 (+cu126) | deep-learning framework              | the neural network inside the agent; runs on the GPU in training, CPU in evaluation                                                                                                                        | 1, 2, 3    |
| Stable-Baselines3 — PPO     | 2.9.0           | RL algorithm library                 | trains the agent ( <code>train_agent.py</code> ) and loads the brain for the dashboard and evaluation; the dashboard draws each move from the loaded policy, so every run takes a new route, on both games | 1, 2, 3    |
| CnnPolicy                   | SB3             | convolutional policy + value network | reads the 84×84×4 frames and chooses one of 10 actions                                                                                                                                                     | 1, 2, 3    |
| SubprocVecEnv / DummyVecEnv | SB3             | vectorised environments              | 8 parallel game workers in training; a single env in the dashboard                                                                                                                                         | 1, 2       |
| SB3 callbacks (custom)      | project         | hooks into training                  | checkpoints with coverage, anchor-KL consolidation, stagnation watch, value warm-up ( <code>training/callbacks.py</code> )                                                                                 | 1, 2       |
| Anchor-KL consolidation     | project         | keeps a skill from being forgotten   | after each update, pulls the policy back toward a frozen healthy brain if $KL > 0.08$                                                                                                                      | 2          |
| cloudpickle                 | 3.1.2           | serialisation                        | saving/loading checkpoints and passing objects to worker processes                                                                                                                                         | 1, 2, 3    |
| TensorBoard (optional)      | ≥ 2.18          | training-curve viewer                | only if present; training runs fine without it                                                                                                                                                             | 1, 2       |
| The brains                  | —               | trained models                       | <code>mario_brain_checkpoint.zip</code> (6M, Objective 1) and <code>glitch_hunter_main_brain.zip</code> (16M, Objectives 2–3), frozen and hash-checked                                                     | 1, 2, 3    |

## Numerical computing, data and storage

| Technology          | Version                              | What it is                      | Where it is used                                                                                               | Objectives |
|---------------------|--------------------------------------|---------------------------------|----------------------------------------------------------------------------------------------------------------|------------|
| NumPy               | 2.5.2                                | arrays and maths                | observations, the coverage bitmap, the testable mask, jump arcs, evaluation statistics                         | 1, 2, 3    |
| .npz files          | NumPy                                | compressed array files          | coverage state, reachability mask, jump arcs, anchor states ( exploration_data/ ), loaded with pickle disabled | 2          |
| Shared memory       | stdlib multiprocessing.shared_memory | memory shared between processes | one global coverage map written by all 8 training workers                                                      | 2          |
| JSON                | stdlib                               | structured records              | incident records, manifests, evaluation results, artifacts.json, INJECTED_BUGS.json, level1_design.json        | 1, 2, 3    |
| JSON Lines (.jsonl) | stdlib                               | append-only logs                | coverage audit trail, reward telemetry, incident occurrences, capture failures                                 | 2, 3       |
| SHA-256 (hashlib)   | stdlib                               | file fingerprints               | artifacts.json protects the brains and masks; incidents and game variants are identified by hash               | 2, 3       |
| ZIP                 | stdlib zipfile                       | archives                        | model checkpoints (SB3 format), incident context frames, the "Download all" evidence bundle                    | 1, 2, 3    |

## Exploration and QA logic (the project's own components)

Built in-house on top of the libraries above.

| Component        | Location                                         | What it does                                                                              | Objectives |
|------------------|--------------------------------------------------|-------------------------------------------------------------------------------------------|------------|
| Game environment | custom_mario_env.py                              | drives the game one frame at a time, reports Mario's position, speed, deaths and glitches | 1, 2, 3    |
| Reward wrapper   | agent_logic.py, rewards/legacy.py, rewards/qa.py | completion reward (Objective 1); coverage + phase-gated QA reward (Objective 2)           | 1, 2       |

| Component                            | Location                                                                 | What it does                                                                                           | Objectives |
|--------------------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------|
| Spatial coverage                     | <code>exploration/coverage.py</code>                                     | 1-px world bitmap of everywhere Mario has been, persistent across episodes                             | 2          |
| Reachability / testable mask         | <code>exploration/reachability.py</code>                                 | which pixels Mario can physically reach (3 methods + real jump arcs) — the coverage denominator        | 2          |
| Episode lifecycle                    | <code>exploration/lifecycle.py</code>                                    | EXPLORE → COMPLETE only on evidence; safety reset on real stagnation, never on time                    | 2          |
| Completion evaluation                | <code>evaluation/completion.py</code>                                    | the frozen 500-episode protocol that decides HEALTHY / WARNING / REGRESSED                             | 1, 2       |
| Game variants                        | <code>reporting/variants.py</code>                                       | identifies clean vs bugged game by tree hash; every bug change must be declared and marked             | 3          |
| Collision and jump-physics detectors | <code>reporting/collision_invariants.py</code>                           | generic rules: clipping into solids, blocked by nothing, hit or stomp without contact, impossible jump | 3          |
| Level design reference               | <code>reporting/level_design.py</code> , <code>level1_design.json</code> | the drawn ground, pipes and steps the detectors compare against                                        | 3          |

## Image and video processing

| Technology             | Version  | What it is        | Where it is used                                                                                                         | Objectives |
|------------------------|----------|-------------------|--------------------------------------------------------------------------------------------------------------------------|------------|
| OpenCV (opencv-python) | 5.0.0.93 | image processing  | resizes each game frame to 480×360 and JPEG-encodes it for the live stream; PNG-encodes the exact trigger frame of a bug | 1, 3       |
| Pillow                 | 12.3.0   | image library     | builds the context GIF of each incident and the finish GIF of each clean run, and prepares images for the PDF report     | 3          |
| Pygame surfaces        | 2.6.1    | raw frame capture | the full-resolution frame copied at the exact moment a detector fires                                                    | 3          |

## Bug evidence and reporting

| Technology / component           | Version                                          | What it is                                           | Where it is used                                                                                                                                                                 | Objectives |
|----------------------------------|--------------------------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| fpdf2                            | 2.8.8                                            | PDF generator                                        | the PDF bug report of every incident ( <code>reporting/render.py</code> ) and the run report of every clean run that reaches the castle ( <code>reporting/run_report.py</code> ) | 3          |
| Markdown                         | —                                                | text report format                                   | the Markdown bug report of every incident, and of every clean-run report                                                                                                         | 3          |
| Incident pipeline                | <code>reporting/pipeline.py</code>               | capture → store → render → replay, in the background | turns each detection into a complete incident; retries and recovers after a crash                                                                                                | 3          |
| Incident store                   | <code>reporting/store.py</code>                  | evidence on disk                                     | one read-only folder per incident with a hash manifest; atomic writes; never overwritten                                                                                         | 3          |
| Clean-run report                 | <code>reporting/run_report.py</code>             | the "No Bugs Found" report                           | written when a clean-game run reaches the castle: run record, final frame, GIF, Markdown, PDF, downloadable as one zip                                                           | 3          |
| Incident schema                  | <code>reporting/schema.py</code>                 | the standard bug record                              | ids, timestamps, deterministic JSON, validation                                                                                                                                  | 3          |
| Severity, confidence, duplicates | <code>reporting/analysis.py</code>               | classification                                       | severity per bug type, detector confidence, same-bug matching and occurrence counts                                                                                              | 3          |
| Replay (reproduction)            | <code>reporting/reproduce.py</code> + subprocess | re-runs the episode headless                         | proves the bug happens again on the same frame: reproduced / not reproduced                                                                                                      | 3          |
| Provenance                       | <code>reporting/provenance.py</code>             | identity of brain, game and code                     | every report says which brain and which game produced it                                                                                                                         | 3          |
| Maintenance tool                 | <code>tools/incidents.py</code>                  | command line                                         | list, show, verify, re-render, re-run replay                                                                                                                                     | 3          |

## Web dashboard: backend

| Technology / component | Version                           | What it is                      | Where it is used                                                                                                                                                                                                                                                                                                                                                                     | Objectives |
|------------------------|-----------------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Flask                  | 3.1.3                             | web framework                   | <code>app.py</code> : serves the page and the REST API: <code>/api/status</code> (state + live status), <code>/api/project</code> (the project's facts), <code>/api/incidents</code> (session, or <code>?all=1</code> ), <code>/api/incidents/&lt;id&gt;</code> , <code>/api/runs</code> (every saved run report), <code>/healthz</code> , and the evidence and run-report downloads | 1, 2, 3    |
| Flask-SocketIO         | 5.6.1                             | real-time messaging             | pushes video frames, log lines, the live status (10 times a second), bug-found, run-finished and report-ready events; receives start, stop, reset, game switch and page-opened (on python-socketio 5.17.0)                                                                                                                                                                           | 1, 2, 3    |
| Threading mode         | Flask-SocketIO                    | concurrency model               | real OS threads (eventlet deliberately not used)                                                                                                                                                                                                                                                                                                                                     | 1, 2, 3    |
| Werkzeug server        | 3.1.8 (via Flask)                 | local HTTP server               | listens on 127.0.0.1:5000 only by default; no debug console                                                                                                                                                                                                                                                                                                                          | 1, 2, 3    |
| Game thread service    | <code>dashboard_service.py</code> | one thread owns the game window | Start / Pause / Resume / Reset / game switch, 30 steps a second, pause on every bug, stop at the end of a clean-game run, forwards the live status every third step; a page refresh is a Reset                                                                                                                                                                                       | 1, 3       |
| Dashboard backend      | <code>dashboard_backend.py</code> | env + brain + frame stream      | loads the approved brain (hash-checked), plays, captures incidents, writes clean-run reports, and sends each step's live status: AI mode (EXPLORE / COMPLETE), action, run and step, level progress, coverage of the reachable level (the brain's own map plus new pixels); counts the brain's parameters                                                                            | 1, 2, 3    |

| Technology / component | Version                                                                              | What it is                  | Where it is used                                                                                                                                                                                                                                                                                                             | Objectives |
|------------------------|--------------------------------------------------------------------------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Project facts          | <code>dashboard_facts.py</code>                                                      | the Overview page's numbers | <code>/api/project</code> : reads <code>FINAL_OBJECTIVE2.json</code> , the 6M baseline, <code>INJECTED_BUGS.json</code> , the detector list, the incident and run-report stores, git, and pytest's own test count (collected once, in a low-priority background process); anything it cannot read is left out, never guessed | 3          |
| Safe file serving      | <code>reporting/store.py</code> and <code>reporting/run_report.py</code> allow-lists | security                    | only known incident / run-report ids and file names can be downloaded; path tricks get 404; <code>/api/runs</code> and <code>/api/project</code> only read                                                                                                                                                                   | 3          |

## Web dashboard: frontend

| Technology             | Version                                      | What it is                          | Where it is used                                                                                                                                                                                                                                                                                                                                                                                                           | Objectives |
|------------------------|----------------------------------------------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| HTML5 (Jinja template) | Jinja2 3.1.6                                 | page structure                      | <code>web/templates/index.html</code> : four tabs (Overview, Live Testing, How It Works, Bug History), game chooser, controls with an "i" button for the 10 actions, live video (NO SIGNAL until testing starts), Live status with its five-step progress list, Bug Tracker, detail windows (dialog elements)                                                                                                              | 1, 2, 3    |
| CSS3                   | —                                            | styling                             | <code>web/static/css/style.css</code> : retro arcade theme (black and green, a pixel font, CRT screen lines, small animations) with colours that carry meaning (running, healthy, warning, bug), responsive grid; the font is bundled, so it works offline                                                                                                                                                                 | 1, 2, 3    |
| Vanilla JavaScript     | ES2017+                                      | page behaviour, no framework        | <code>web/static/js/main.js</code> : shows only what the server reports: tabs, controls, live video, Live status with its five-step progress list, the Bug found (with where to fix it) and run-result cards, the game screen holding its last frame when testing stops, Bug Tracker, Bug History tables with Delete and Clear history (asked first), detail windows, reset on page refresh, game switch; inline SVG icons | 1, 2, 3    |
| Socket.IO JS client    | vendored ( <code>web/static/vendor/</code> ) | browser side of the live connection | receives frames and events; kept locally so the dashboard works offline                                                                                                                                                                                                                                                                                                                                                    | 1, 2, 3    |



| Technology          | Version                                               | What it is                         | Where it is used                                                                                                    | Objectives |
|---------------------|-------------------------------------------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------------|------------|
| Press Start 2P font | vendored<br>( <code>web/static/vendor/fonts/</code> ) | pixel font for headings and labels | the retro arcade look; kept locally so the dashboard works offline (SIL Open Font License 1.1)                      | 3          |
| Fetch API           | browser                                               | REST calls                         | loads <code>/api/status</code> , <code>/api/incidents</code> , <code>/api/project</code> and <code>/api/runs</code> | 3          |

## Testing, code quality and CI/CD

| Technology            | Version                                          | What it is                  | Where it is used                                                                                                                                                                                                                                         | Objectives |
|-----------------------|--------------------------------------------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| pytest                | 9.1.1                                            | test framework              | 888 automated tests in <code>tests/</code> (fast ones by default, <code>-m slow</code> for brain and end-to-end)                                                                                                                                         | 1, 2, 3    |
| pytest-cov            | 7.1.0                                            | code coverage               | the full gate requires at least 90% (last run 90.68%, on 2026-09-28, measured to two decimals)                                                                                                                                                           | 2, 3       |
| Ruff                  | 0.16.6                                           | linter                      | style, bugs and security (bandit) rules on every run                                                                                                                                                                                                     | 2, 3       |
| mypy                  | 2.3.1                                            | static type checker         | strict mode, checked for both Windows and Linux                                                                                                                                                                                                          | 2, 3       |
| pre-commit            | 4.6.2                                            | git hooks                   | runs checks before each commit                                                                                                                                                                                                                           | 2, 3       |
| Quality gate          | <code>tools/check.py</code>                      | one command for all checks  | lint → types → tests → artifact hashes ( <code>--full</code> adds slow tests + coverage floor)                                                                                                                                                           | 2, 3       |
| Artifact verification | <code>tools/verify_artifacts.py</code>           | integrity check             | re-hashes the protected brains, masks and baseline against <code>artifacts.json</code>                                                                                                                                                                   | 2, 3       |
| End-to-end validation | <code>tools/validate_incident_pipeline.py</code> | 37-check pipeline proof     | runs the real dashboard stack headless and checks every reporting stage                                                                                                                                                                                  | 3          |
| Git + GitHub          | —                                                | version control and hosting | source history and the remote repository                                                                                                                                                                                                                 | 1, 2, 3    |
| GitHub Actions        | <code>.github/workflows/ci.yml</code>            | continuous integration      | lint, types and tests (86% coverage floor) on Ubuntu, Windows and macOS on every push, and the one-click setup on clean Windows, Linux and macOS machines, ending with the dashboard up on the approved brain; on Linux, a browser test of the dashboard | 2, 3       |

| Technology | Version | What it is         | Where it is used                                                                                                   | Objectives |
|------------|---------|--------------------|--------------------------------------------------------------------------------------------------------------------|------------|
| Playwright | 1.63.0  | browser automation | drives the dashboard in headless Chromium in CI ( <code>tests/test_dashboard_e2e.py</code> ); installed only there | 3          |

## Tooling, scripts and documentation

| Tool / format                                                                                                                                                                                                         | Where it is used                                                                                                                                                                            | Objectives |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <code>train_agent.py</code>                                                                                                                                                                                           | training, with safe resume ( <code>--resume-from</code> , <code>--dry-run-resume</code> )                                                                                                   | 1, 2       |
| <code>tools/evaluate_completion.py</code>                                                                                                                                                                             | 500-episode completion test (HEALTHY / WARNING / REGRESSED)                                                                                                                                 | 1, 2       |
| <code>tools/build_reachability.py</code> , <code>collect_jump_arcs.py</code> , <code>bootstrap_coverage.py</code> , <code>migrate_coverage.py</code> , <code>build_anchor_set.py</code> , <code>calibrate_*.py</code> | one-time setup and measurement for the QA campaign                                                                                                                                          | 2          |
| <code>tools/build_level_design.py</code>                                                                                                                                                                              | writes the level-design reference from the clean game                                                                                                                                       | 3          |
| <code>run_dashboard.bat</code>                                                                                                                                                                                        | double-click to start the dashboard and open the browser                                                                                                                                    | 3          |
| Markdown docs                                                                                                                                                                                                         | <code>README.md</code> , <code>docs/OBJECTIVE1-3.md</code> , <code>docs/ARCHITECTURE.md</code> , <code>docs/PERFORMANCE.md</code> , <code>SECURITY.md</code> , <code>CONTRIBUTING.md</code> | 1, 2, 3    |
| Mermaid diagrams                                                                                                                                                                                                      | the architecture diagram in <code>docs/ARCHITECTURE.md</code>                                                                                                                               | 2, 3       |

## Project map: every folder and file

Every folder and file in the project, tracked in git or not, with what it holds and what uses it. "In git" says whether the file is committed; gitignored files exist only on the machine that made or installed them. Large generated sets (the Python environment, caches) are described as a folder.

### Folders at a glance

| Folder                | What it holds                                                                       | Where it is used                                     | In git |
|-----------------------|-------------------------------------------------------------------------------------|------------------------------------------------------|--------|
| <code>common/</code>  | small shared helpers: logging, atomic file writes, hashes, tool start-up            | every entry point and tool                           | yes    |
| <code>rewards/</code> | the two reward functions: completion (Objective 1) and QA exploration (Objective 2) | <code>agent_logic.py</code> during training and play | yes    |

| Folder                                  | What it holds                                                                                                                                                             | Where it is used                                                                                        | In git                                                            |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <code>training/</code>                  | Stable-Baselines3 callbacks, checkpoint helpers, the one-off value-head reset                                                                                             | <code>train_agent.py</code>                                                                             | yes                                                               |
| <code>evaluation/</code>                | the 500-episode completion protocol, Level-1 verification, the frozen 6M baseline                                                                                         | training resume checks, <code>tools/evaluate_completion.py</code> , <code>tools/verify_level1.py</code> | yes ( <code>results/</code> is not)                               |
| <code>exploration/</code>               | <code>config.py</code> (every tuned value), spatial coverage, reachability mask, episode lifecycle, level completion                                                      | the environment, reward wrapper, training, dashboard                                                    | yes                                                               |
| <code>reporting/</code>                 | Objective 3: detectors, incident pipeline, store, reports, replay, run reports, game variants                                                                             | dashboard, tests, <code>tools/incidents.py</code>                                                       | yes                                                               |
| <code>web/ (templates/, static/)</code> | the dashboard web page: HTML, CSS, JavaScript, bundled Socket.IO client and pixel font                                                                                    | served by <code>app.py</code>                                                                           | yes                                                               |
| <code>docs/</code>                      | Objective 1-3 summaries, architecture, performance notes, contributing, security, third-party notices, the Tech Stack and Panel Q&A PDFs, and the README images (assets/) | readers; linked from <code>README.md</code>                                                             | yes                                                               |
| <code>tools/</code>                     | command-line tools: quality gate, evaluation, data builders, incident tools, installer                                                                                    | run by hand and by <code>tools/check.py</code> / tests                                                  | yes                                                               |
| <code>tests/</code>                     | 888 automated tests and their helpers                                                                                                                                     | <code>pytest</code> , <code>tools/check.py</code> , CI                                                  | yes                                                               |
| <code>.github/</code>                   | the CI workflow                                                                                                                                                           | GitHub Actions on every push                                                                            | yes                                                               |
| <code>games/mario_clean/</code>         | the original Mario clone (code, graphics, music, sounds), unmodified and hash-pinned                                                                                      | the game every brain is trained and measured on                                                         | yes                                                               |
| <code>games/mario_bugged/</code>        | a copy of the clone with the 6 declared benchmark bugs                                                                                                                    | the bug-detection target, picked in the dashboard                                                       | yes                                                               |
| <code>checkpoints_qa/</code>            | the final 16M brain's closure record, evidence, and the 6.03M anchor checkpoint                                                                                           | dashboard approval check, retraining with anchor-KL, <code>artifacts.json</code>                        | only <code>final_objective2_16000000/FINAL_OBJECTIVE2.json</code> |
| <code>exploration_data/</code>          | measured data: reachability mask, jump arcs, observed reach, coverage bootstrap, anchor states                                                                            | QA reward, coverage, anchor-KL training                                                                 | only <code>reachable_mask.npz</code>                              |

| Folder                  | What it holds                                                                                                                                                                                                                                  | Where it is used                                     | In git             |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|--------------------|
| <code>generated/</code> | everything written as the project runs: incidents/ (bug evidence), run_reports/ (clean runs), coverage_audits/, calibration_runs/, logs/ (training), and cache/ (ruff, mypy, pytest, coverage); the project writes no <code>__pycache__</code> | the dashboard, the tools, training, the quality gate | no                 |
| <code>venv_gpu/</code>  | the Python environment: Python 3.12 and every installed library (about 4.5 GB)                                                                                                                                                                 | every command                                        | no                 |
| <code>.git/</code>      | the repository's full history                                                                                                                                                                                                                  | git                                                  | (git's own folder) |

## Files in the project root

| File                              | What it is and what it holds                                                                                                                                                                                                     | Where it is used                                                        | In git |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|--------|
| <code>app.py</code>               | the dashboard server (Flask + Socket.IO): page, REST API ( <code>/api/status</code> , <code>/api/project</code> , <code>/api/incidents</code> , <code>/api/runs</code> ), report and evidence downloads, socket events           | started by <code>python app.py</code> or <code>run_dashboard.bat</code> | yes    |
| <code>dashboard_service.py</code> | the single game thread: Start / Pause / Reset / game switch, pause on bugs, stop at a clean run's end, step pacing, the live status sent to the page                                                                             | <code>app.py</code>                                                     | yes    |
| <code>dashboard_backend.py</code> | loads the approved brain, runs the game, streams frames and the live status, picks each move (a new route every run; a stuck run ended early), captures incidents, writes run reports                                            | <code>dashboard_service.py</code> , <code>app.py</code>                 | yes    |
| <code>dashboard_facts.py</code>   | the Overview page's facts, read from the project's own files: the objectives' results, detectors, benchmark bugs and evidence on disk (its API also reports code size, commits and the test count, which the page does not show) | <code>app.py</code> ( <code>/api/project</code> )                       | yes    |
| <code>desktop.py</code>           | <code>run_dashboard.bat</code> 's helpers: full speed on battery (power mode, no throttling), centred terminal and browser                                                                                                       | <code>run_dashboard.bat</code> , <code>app.py --desktop</code>          | yes    |

| File                                               | What it is and what it holds                                                                                                                                                                                                       | Where it is used                                                                      | In git                                                                     |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>game_window.py</code>                        | Windows window calls for the game window: centre, show, hide, minimise to the taskbar                                                                                                                                              | <code>custom_mario_env.py</code> ,<br><code>desktop.py</code>                         | yes                                                                        |
| <code>custom_mario_env.py</code>                   | the game as a Gymnasium environment: actions, observations, info, built-in detectors, the game window's life cycle                                                                                                                 | training, dashboard, evaluation, tests, tools                                         | yes                                                                        |
| <code>agent_logic.py</code>                        | <code>GlitchHunterWrapper</code> : coverage recording, the episode lifecycle and the reward                                                                                                                                        | training, dashboard, evaluation                                                       | yes                                                                        |
| <code>train_agent.py</code>                        | training for Objectives 1 and 2, with safe resume                                                                                                                                                                                  | run by hand ( <code>python train_agent.py</code> )                                    | yes                                                                        |
| <code>run_dashboard.bat</code>                     | Windows double-click launcher for the dashboard                                                                                                                                                                                    | the user                                                                              | yes                                                                        |
| <code>setup.bat</code>                             | Windows one-click setup: finds (or installs with winget) Python 3.12, creates <code>venv_gpu</code> , installs the CPU build of PyTorch (gpu: the CUDA build) and the libraries, checks the brain files, then starts the dashboard | a new user, once after cloning; CI ( <code>setup.bat norun</code> )                   | yes                                                                        |
| <code>setup.sh</code>                              | the same one-command setup for Linux and macOS                                                                                                                                                                                     | <code>bash setup.sh</code> ; CI ( <code>bash setup.sh norun</code> )                  | yes                                                                        |
| <code>mario_brain_checkpoint.zip</code>            | the 6M brain (Objective 1), 20 MB: the fallback brain                                                                                                                                                                              | dashboard when the 16M brain is absent; baseline for evaluation                       | no (git-ignored since Sep 29, 2026; earlier commits hold it)               |
| <code>glitch_hunter_main_brain.zip</code>          | the final 16M brain (Objectives 2-3), 20 MB, hash-checked                                                                                                                                                                          | dashboard's default brain; a resume candidate for training                            | yes (also a release asset that <code>tools/final_brain.py</code> installs) |
| <code>glitch_hunter_main_brain_coverage.npz</code> | the 16M brain's coverage map (where it has been in the level)                                                                                                                                                                      | dashboard's QA reward, coverage maps                                                  | yes                                                                        |
| <code>artifacts.json</code>                        | SHA-256 of every protected brain, mask, baseline and evidence file                                                                                                                                                                 | <code>tools/verify_artifacts.py</code> ,<br><code>tools/final_brain.py</code> , tests | yes                                                                        |
| <code>pyproject.toml</code>                        | dependencies and the settings for ruff, mypy, pytest and coverage                                                                                                                                                                  | <code>uv sync</code> , every quality tool, CI                                         | yes                                                                        |

| File                                 | What it is and what it holds                                                        | Where it is used                                  | In git |
|--------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------|--------|
| <code>uv.lock</code>                 | exact locked versions of every dependency                                           | <code>uv sync --active</code>                     | yes    |
| <code>requirements.txt</code>        | the same dependencies for plain pip, with the install steps                         | <code>pip install -r requirements.txt</code> , CI | yes    |
| <code>README.md</code>               | the main guide: overview, installation (one-click setup first), how to run, results | readers, GitHub front page                        | yes    |
| <code>LICENSE</code>                 | MIT licence for the project's own work, and what it does not cover                  | readers                                           | yes    |
| <code>.gitignore</code>              | what git must never commit, and why                                                 | git                                               | yes    |
| <code>.gitattributes</code>          | line-ending and binary rules                                                        | git                                               | yes    |
| <code>.pre-commit-config.yaml</code> | optional checks run on every commit (ruff, mypy, file hygiene)                      | <code>pre-commit</code>                           | yes    |

## Core source packages

All tracked in git; each folder's `__init__.py` marks it as a Python package and says what it is for.

| File                                 | What it is and what it holds                                                              | Where it is used                               |
|--------------------------------------|-------------------------------------------------------------------------------------------|------------------------------------------------|
| <code>common/cli.py</code>           | the start-up every tool shares: project root on the path, headless SDL, logging           | every script in <code>tools/</code>            |
| <code>common/fileio.py</code>        | atomic file writes, SHA-256 hashes, read-only protection for artifacts                    | training, reporting, tools                     |
| <code>common/logging_setup.py</code> | one logging configuration and the progress log                                            | every entry point                              |
| <code>rewards/legacy.py</code>       | the completion reward, unchanged from the 6M training run                                 | <code>agent_logic.py</code> in completion mode |
| <code>rewards/qa.py</code>           | the QA exploration reward: new coverage, phase-gated completion bonus                     | <code>agent_logic.py</code> in QA mode         |
| <code>rewards/shared.py</code>       | state and shaping both rewards share, incl. the adaptive death memory                     | both reward modes                              |
| <code>training/callbacks.py</code>   | the training callbacks: checkpoints with coverage, anchor-KL, stagnation watch, telemetry | <code>train_agent.py</code>                    |
| <code>training/checkpoints.py</code> | reads a checkpoint's step count without loading it; retention verdicts for resume         | <code>train_agent.py</code>                    |
| <code>training/value_head.py</code>  | the one-off critic reset when QA training starts from the 6M brain                        | <code>train_agent.py</code>                    |

| File                                                | What it is and what it holds                                                        | Where it is used                                               |
|-----------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------|
| <code>evaluation/completion.py</code>               | the frozen 500-episode test: can a brain still finish Level 1-1?                    | <code>tools/evaluate_completion.py</code> , resume checks      |
| <code>evaluation/level1_verification.py</code>      | decides whether a Level-1 snapshot is the final brain                               | <code>tools/verify_level1.py</code>                            |
| <code>evaluation/completion_baseline_6M.json</code> | the 6M brain's frozen completion result (46.8%) that others are compared with       | <code>evaluation/completion.py</code> ; hash-protected         |
| <code>exploration/config.py</code>                  | every tuned value (rewards, thresholds, paths), each with the measurement behind it | nearly every module                                            |
| <code>exploration/coverage.py</code>                | the world-space coverage bitmap, shared by all training workers                     | reward wrapper, training, dashboard, tools                     |
| <code>exploration/reachability.py</code>            | the testable-pixel mask: which pixels Mario can physically reach                    | coverage percentages, <code>tools/build_reachability.py</code> |
| <code>exploration/lifecycle.py</code>               | the episode lifecycle: EXPLORE to COMPLETE, and how an episode ends                 | reward wrapper, dashboard                                      |
| <code>exploration/level_completion.py</code>        | when Level 1 counts as fully covered; coverage maps and region lists                | training, <code>tools/remaining_coverage_map.py</code>         |

`evaluation/results/` (not in git) holds one completion-test result per evaluated brain; the 16M brain's result is hash-protected and read when choosing what to resume.

## reporting/ (Objective 3)

All tracked in git.

| File                                 | What it is and what it holds                                                                                       | Where it is used                                                                        |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>__init__.py</code>             | package marker: turns a detected anomaly into a reviewable incident                                                | -                                                                                       |
| <code>events.py</code>               | the <code>Detection</code> a detector produces, and the synthetic test probe                                       | environment, pipeline                                                                   |
| <code>collision_invariants.py</code> | the generic bug detectors: clipping into solids, blocked by nothing, hit or stomp without contact, impossible jump | <code>custom_mario_env.py</code>                                                        |
| <code>level_design.py</code>         | loads Level 1-1 as designed, for the detectors to compare against                                                  | <code>collision_invariants.py</code>                                                    |
| <code>level1_design.json</code>      | the drawn ground, pipes and steps (37 solids) measured from the clean game                                         | <code>level_design.py</code> ; rebuilt only by <code>tools/build_level_design.py</code> |
| <code>pipeline.py</code>             | capture, then GIF, reports and replay in the background; duplicate counting; session view                          | dashboard backend                                                                       |

| File                       | What it is and what it holds                                                                                                                                                                                                          | Where it is used                                                              |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>evidence.py</code>   | writes the raw evidence: trigger frame, context frames, trajectory, the incident record                                                                                                                                               | <code>pipeline.py</code>                                                      |
| <code>schema.py</code>     | the incident record's format, ids and validation                                                                                                                                                                                      | <code>pipeline</code> , <code>store</code> , <code>tools</code>               |
| <code>store.py</code>      | the incident folders on disk: atomic, read-only, never overwritten; safe file lookup                                                                                                                                                  | <code>pipeline</code> , <code>app.py</code> , <code>tools/incidents.py</code> |
| <code>analysis.py</code>   | severity, detector confidence and the duplicate fingerprint                                                                                                                                                                           | <code>pipeline.py</code>                                                      |
| <code>render.py</code>     | an incident's GIF, Markdown report and PDF report                                                                                                                                                                                     | <code>pipeline.py</code> , <code>tools/incidents.py</code>                    |
| <code>fix_hint.py</code>   | an incident's "Where to fix it": the lines in the game's code that cause it and the exact edit for each (delete / change to), worked out from the incident's record and the game's source (read with ast); a lead, not a proven cause | <code>render.py</code> , <code>pipeline.py</code> (the dashboard's bug card)  |
| <code>reproduce.py</code>  | replays an incident's episode in a separate process and says whether it happened again                                                                                                                                                | <code>pipeline.py</code>                                                      |
| <code>provenance.py</code> | which brain, game and code produced an incident                                                                                                                                                                                       | dashboard backend, <code>pipeline</code>                                      |
| <code>variants.py</code>   | the clean and bugged game variants, their tree hashes and declared bugs                                                                                                                                                               | environment, <code>provenance</code> , <code>tests</code>                     |
| <code>run_report.py</code> | the clean game's "No Bugs Found" run report: record, final frame, GIF, Markdown, PDF; the list of every saved run (Bug History, <code>/api/runs</code> )                                                                              | dashboard backend, <code>app.py</code>                                        |

## Dashboard page, images, docs and CI

All tracked in git.

| File                                            | What it is and what it holds                                                                                                                                                  | Where it is used                                |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <code>web/templates/index.html</code>           | the dashboard page: the four tabs, game chooser, controls, video, Live status with its five-step progress list, Bug Tracker, Bug History, detail windows                      | served by <code>app.py</code> at <code>/</code> |
| <code>web/static/js/main.js</code>              | the page's behaviour: tabs, controls, live video, live status, Bug found and run-result cards, Bug Tracker, Bug History, detail windows, reset (also on refresh), game switch | loaded by <code>index.html</code>               |
| <code>web/static/css/style.css</code>           | the page's retro arcade theme, status colours and layout                                                                                                                      | loaded by <code>index.html</code>               |
| <code>web/static/vendor/socket.io.min.js</code> | the Socket.IO browser client (v4.7.2), bundled so the page works offline                                                                                                      | loaded by <code>index.html</code>               |



| File                              | What it is and what it holds                                                                                                                                                                                          | Where it is used                    |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| web/static/vendor/fonts/          | the Press Start 2P pixel font and its license (OFL.txt), bundled so the page works offline                                                                                                                            | loaded by style.css                 |
| docs/assets/dashboard.png         | screenshot of the Live Testing tab while testing the clean game                                                                                                                                                       | README.md                           |
| docs/assets/bug-found.png         | screenshot of a "Bug found" stop (stair clipping)                                                                                                                                                                     | README.md                           |
| docs/assets/bug-stair-clip.gif    | recording of the stair-clipping bug being caught                                                                                                                                                                      | README.md                           |
| docs/OBJECTIVE1.md                | Objective 1: the 6M brain, how it was trained, its results                                                                                                                                                            | readers; README.md                  |
| docs/OBJECTIVE2.md                | Objective 2: QA exploration, audits, the final 16M brain                                                                                                                                                              | readers; README.md                  |
| docs/OBJECTIVE3.md                | Objective 3: detectors, reporting pipeline, benchmark bugs, run reports                                                                                                                                               | readers; README.md                  |
| docs/ARCHITECTURE.md              | components, invariants, protected artifacts, where to change what                                                                                                                                                     | readers; README.md                  |
| docs/PERFORMANCE.md               | time and memory per training worker, and how to measure them                                                                                                                                                          | readers; README.md                  |
| docs/CONTRIBUTING.md              | how to develop: set-up, quality gate, code rules, commits, releasing the brain                                                                                                                                        | contributors; README.md             |
| docs/SECURITY.md                  | what the dashboard exposes, what run_dashboard.bat and the setup scripts change, dependency audit                                                                                                                     | readers; README.md                  |
| docs/THIRD_PARTY_NOTICES.md       | terms of the Mario clone, Nintendo assets and every library                                                                                                                                                           | readers; LICENSE, README.md         |
| docs/Glitch_Hunter_Tech_Stack.pdf | this document as a PDF                                                                                                                                                                                                | readers; README.md                  |
| docs/Glitch_Hunter_Panel_QA.pdf   | 154 questions a presentation panel may ask about the project, with simple-English answers (purpose, AI, the three objectives, the dashboard and its Start testing code path, making it generic, related work, limits) | presenters, before a viva or review |
| .github/workflows/ci.yml          | continuous integration: lint, types and tests on Ubuntu, Windows and macOS; the one-click setup on Ubuntu, Windows and macOS; a browser test of the dashboard                                                         | GitHub Actions on every push        |

## tools/ (command-line tools)

All tracked in git; each is run by hand as `python tools/<name>.py`, and `tests/test_tools_cli.py` checks every one starts and documents itself.

| File                                       | What it does                                                                                                                                                          | What it writes                                                   |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <code>README.md</code>                     | the guide to every tool below                                                                                                                                         | -                                                                |
| <code>check.py</code>                      | every quality gate CI runs: ruff, mypy, tests, artifact hashes ( <code>--full</code> : slow tests + coverage floor)                                                   | nothing                                                          |
| <code>final_brain.py</code>                | for a checkout without them, installs the final 16M brain's files from the GitHub Release (verified against <code>artifacts.json</code> ), or builds that release zip | the four final-brain files                                       |
| <code>verify_artifacts.py</code>           | re-hashes every protected artifact against <code>artifacts.json</code>                                                                                                | nothing ( <code>--record</code> rewrites the manifest)           |
| <code>benchmark_step.py</code>             | milliseconds per agent step, and where the time goes                                                                                                                  | nothing                                                          |
| <code>evaluate_completion.py</code>        | can a brain still finish Level 1-1? HEALTHY / WARNING / REGRESSED against the 6M baseline                                                                             | <code>evaluation/results/</code>                                 |
| <code>verify_level1.py</code>              | checks a Level-1 snapshot before it counts as the final brain                                                                                                         | a verification JSON beside the snapshot                          |
| <code>remaining_coverage_map.py</code>     | where the uncovered reachable pixels are, as a picture and a region list                                                                                              | <code>generated/coverage_audits/</code>                          |
| <code>incidents.py</code>                  | the incident store: list, show, verify, re-render, reproduce, recover                                                                                                 | only new versioned files, never overwrites                       |
| <code>validate_incident_pipeline.py</code> | 37-check end-to-end proof of the reporting pipeline on the real dashboard stack                                                                                       | its own store under <code>generated/incidents/validation/</code> |
| <code>build_level_design.py</code>         | writes Level 1-1's drawn solids from the clean game                                                                                                                   | <code>reporting/level1_design.json</code>                        |
| <code>collect_jump_arcs.py</code>          | records the engine's real jump arcs                                                                                                                                   | <code>exploration_data/jump_arcs.npz</code>                      |
| <code>build_reachability.py</code>         | builds the reachable-pixel mask (the coverage denominator)                                                                                                            | <code>exploration_data/reachable_mask.npz</code>                 |
| <code>build_anchor_set.py</code>           | records the anchor states a healthy brain sees, for anchor-KL training                                                                                                | <code>exploration_data/anchor_states.npz</code>                  |
| <code>migrate_coverage.py</code>           | re-stamps a coverage file to a rebuilt mask, losslessly                                                                                                               | a new coverage file                                              |

| File                                             | What it does                                                                     | What it writes                                        |
|--------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------|
| <code>bootstrap_coverage.py</code>               | seeds the coverage map with the 6M brain's usual routes                          | <code>exploration_data/coverage_bootstrap*.npz</code> |
| <code>calibrate_reward.py</code>                 | solves the QA reward's scale against the completion reward                       | a value in <code>exploration/config.py</code>         |
| <code>calibrate_phase_reward.py</code>           | measures every QA reward channel on real runs (source of the phase-gated values) | <code>generated/calibration_runs/</code>              |
| <code>objective2_evidence/jump_physics.py</code> | engine ground truth for the 172 px obstacles (tall pipes, staircase top)         | nothing                                               |
| <code>objective2_evidence/runup.py</code>        | the minimum run-up for a full-height jump                                        | nothing                                               |
| <code>objective2_evidence/well_escape.py</code>  | exhaustive search: is the staircase well a soft-lock?                            | nothing                                               |
| <code>objective2_evidence/well_trace.py</code>   | replays one well-escape attempt frame by frame                                   | nothing                                               |

## tests/

All tracked in git and run by `pytest` (through `tools/check.py` and CI). The first eight are helpers, not tests.

| File                                 | What it holds or checks                                                                                                                  |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>conftest.py</code>             | shared fixtures (the real game environment, temporary folders) for every test                                                            |
| <code>incident_helpers.py</code>     | builds realistic detections and capture context without running the game, and a stand-in brain; used by the incident and dashboard tests |
| <code>mp_workers.py</code>           | worker processes for the multi-process coverage test                                                                                     |
| <code>bug_probe.py</code>            | plays every injected-bug scenario on one game variant; run by <code>test_injected_bugs.py</code>                                         |
| <code>variant_probe.py</code>        | plays one game variant and prints its fingerprint; run by <code>test_game_variants.py</code>                                             |
| <code>bug_incident_run.py</code>     | one dashboard episode through the real reporting pipeline; run by <code>test_bug_incidents.py</code>                                     |
| <code>fix_hint_incidents.json</code> | the eight real incidents of the benchmark bugs, trimmed; read by <code>test_fix_hint.py</code>                                           |
| <code>fix_probe.py</code>            | plays the bug scenarios on a patched copy of the bugged game; run by <code>test_fix_hint.py</code>                                       |
| <code>test_arc_reach.py</code>       | reachability from measured jump arcs instead of a rectangle                                                                              |
| <code>test_artifacts.py</code>       | protected files are still the recorded bytes; the main brain is the approved one                                                         |

| File                                      | What it holds or checks                                                                                                                                                                                                                                            |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_bug_incidents.py</code>        | all six benchmark bugs become reproduced, reported incidents; the clean game reports none                                                                                                                                                                          |
| <code>test_collision_invariants.py</code> | the collision and jump detectors, with a regression test per false alarm                                                                                                                                                                                           |
| <code>test_common.py</code>               | <code>common/</code> : atomic file I/O, logging, tool start-up                                                                                                                                                                                                     |
| <code>test_completion_eval.py</code>      | the 500-episode completion evaluation                                                                                                                                                                                                                              |
| <code>test_completion_actor.py</code>     | the evaluation's action choice, checkpoint record and interval maths, on stand-ins                                                                                                                                                                                 |
| <code>test_concurrency.py</code>          | thread-safety of the dashboard server                                                                                                                                                                                                                              |
| <code>test_coverage.py</code>             | coverage recording, saving, frontier, adaptive target                                                                                                                                                                                                              |
| <code>test_coverage_env.py</code>         | coverage from the real game is in world space                                                                                                                                                                                                                      |
| <code>test_coverage_multiproc.py</code>   | coverage is shared across real OS processes                                                                                                                                                                                                                        |
| <code>test_dashboard_backend.py</code>    | what the browser is sent, the live status, the brain's facts, <code>/api/project</code> , clean-run endings and reports, a new route every run (the bugged game's first keeps the fixed one), stuck runs ended early, refresh = reset, <code>--desktop</code> mode |
| <code>test_dashboard_control.py</code>    | the game thread and window: Start, Pause, X, Reset, game switch, pacing, minimised start, the live status sent every few steps                                                                                                                                     |
| <code>test_dashboard_facts.py</code>      | the Overview's facts: read from the project's own files, never guessed; the test count                                                                                                                                                                             |
| <code>test_dashboard_e2e.py</code>        | the dashboard in a real browser: Start testing, a bug stops it, the card says where and what to fix (skips without Playwright)                                                                                                                                     |
| <code>test_dashboard_incidents.py</code>  | the dashboard side of Objective 3: stops, Bug Tracker, downloads                                                                                                                                                                                                   |
| <code>test_desktop.py</code>              | <code>desktop.py</code> : power-mode guard, restorer, centred windows                                                                                                                                                                                              |
| <code>test_env.py</code>                  | the environment's contract and past-bug regressions                                                                                                                                                                                                                |
| <code>test_episode_lifecycle.py</code>    | the engine timer, EXPLORE to COMPLETE, and how episodes end                                                                                                                                                                                                        |
| <code>test_final_brain.py</code>          | <code>tools/final_brain.py</code> installs the 16M brain intact                                                                                                                                                                                                    |
| <code>test_fix_hint.py</code>             | "Where to fix it" lands on the injected code for every benchmark bug, without reading the bug list, and applying its edits makes every bug's spot play exactly like the clean game                                                                                 |
| <code>test_game_variants.py</code>        | the clean game is untouched; the bugged game differs only where declared                                                                                                                                                                                           |
| <code>test_game_window.py</code>          | show, hide and minimise calls of the game window                                                                                                                                                                                                                   |
| <code>test_glitch_detection.py</code>     | the built-in detectors behind the Bug Tracker                                                                                                                                                                                                                      |
| <code>test_hud_timer.py</code>            | the on-screen TIME box during long QA episodes                                                                                                                                                                                                                     |
| <code>test_incident_capture.py</code>     | capturing evidence from the real game                                                                                                                                                                                                                              |

| File                                       | What it holds or checks                                                                                                        |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>test_incident_pipeline.py</code>     | schema, store, duplicates, reports, without the game                                                                           |
| <code>test_incident_reproduce.py</code>    | replay is honest both ways (reproduced / not reproduced)                                                                       |
| <code>test_incident_tools.py</code>        | <code>tools/incidents.py</code> and the end-to-end validation                                                                  |
| <code>test_injected_bugs.py</code>         | each of the six bugs, proved against the clean game                                                                            |
| <code>test_level1_compliance.py</code>     | what surrounds Level-1 completion                                                                                              |
| <code>test_level_completion.py</code>      | QA training ends on Level-1 coverage, not a step count                                                                         |
| <code>test_migrate_coverage.py</code>      | <code>tools/migrate_coverage.py</code>                                                                                         |
| <code>test_noncoverage_taxonomy.py</code>  | the two false-coverage causes found during Objective 2                                                                         |
| <code>test_objective2_protection.py</code> | Objective 3 never writes into Objective 2's files                                                                              |
| <code>test_phase_reward.py</code>          | the phase-gated QA reward                                                                                                      |
| <code>test_qa_resume.py</code>             | QA checkpoints, resuming, and the 6M brain's safety                                                                            |
| <code>test_quality_gate.py</code>          | the coverage floor is measured to two decimals                                                                                 |
| <code>test_reward_calibration.py</code>    | the calibrated reward and its bookkeeping                                                                                      |
| <code>test_reward_qa.py</code>             | reward shaping in QA mode                                                                                                      |
| <code>test_reward_telemetry.py</code>      | the reward telemetry written during training                                                                                   |
| <code>test_reward_wrapper.py</code>        | reward shaping in completion (legacy) mode                                                                                     |
| <code>test_rewards_death_memory.py</code>  | the adaptive death memory                                                                                                      |
| <code>test_run_report.py</code>            | the "No Bugs Found" run report, its files and downloads, the list of saved runs and <code>/api/runs</code>                     |
| <code>test_setup_scripts.py</code>         | <code>setup.bat</code> and <code>setup.sh</code> install the pinned torch, check every brain file, and keep their line endings |
| <code>test_testable_mask.py</code>         | the coverage denominator never exceeds the real world                                                                          |
| <code>test_tools_cli.py</code>             | every tool starts, parses its options and documents itself                                                                     |
| <code>test_train_agent_stages.py</code>    | <code>train_agent.py</code> : resume, device, callbacks, banner                                                                |
| <code>test_training_callbacks.py</code>    | the training callbacks and checkpoint helpers                                                                                  |
| <code>test_value_head.py</code>            | the one-off value-head reset touches only the critic's output layer, on a tiny stand-in                                        |

## The game: `games/mario_clean/` and `games/mario_bugged/`

Both folders, side by side in `games/`, hold the same third-party Mario clone and are tracked in git. `games/mario_clean/` is never edited (its whole tree is pinned by SHA-256); `games/mario_bugged/` differs only where `INJECTED_BUGS.json` declares a bug. The game code is loaded by `custom_mario_env.py`.

| File (in each folder)                                                                                           | What it is and what it holds                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>VARIANT.md</code>                                                                                         | what this copy is for; in <code>mario_bugged/</code> , the six bugs and the rules for changing it                                                                                                                                         |
| <code>INJECTED_BUGS.json</code> ( <code>mario_bugged/</code> only)                                              | the declaration of every benchmark bug: where, which lines, the pinned diff hash                                                                                                                                                          |
| <code>data/__init__.py</code> , <code>data/components/__init__.py</code> , <code>data/states/__init__.py</code> | package markers                                                                                                                                                                                                                           |
| <code>data/setup.py</code>                                                                                      | starts pygame, opens the game window, loads every image, song and sound                                                                                                                                                                   |
| <code>data/tools.py</code>                                                                                      | <code>Control</code> , the game's main loop and state machine; the image/music/sound loaders                                                                                                                                              |
| <code>data/constants.py</code>                                                                                  | screen size, colours, physics values, state names                                                                                                                                                                                         |
| <code>data/game_sound.py</code>                                                                                 | which music and sound effects play when                                                                                                                                                                                                   |
| <code>data/states/level1.py</code>                                                                              | Level 1-1 itself: builds ground, pipes, steps, bricks and enemies, and runs every frame's collisions (in <code>mario_bugged/</code> , carries five of the six bugs: stair, pipe and ceiling clipping, invisible wall, stomp from too far) |
| <code>data/components/mario.py</code>                                                                           | Mario: movement, jumping, growing, dying, animation (in <code>mario_bugged/</code> , carries the open-sky jump bug)                                                                                                                       |
| <code>data/components/enemies.py</code>                                                                         | Goombas and Koopas                                                                                                                                                                                                                        |
| <code>data/components/bricks.py</code>                                                                          | bricks and the pieces they break into                                                                                                                                                                                                     |
| <code>data/components/coin_box.py</code>                                                                        | ? boxes                                                                                                                                                                                                                                   |
| <code>data/components/coin.py</code> , <code>data/components/flashing_coin.py</code>                            | coins that pop out of boxes; the blinking coin in the score bar                                                                                                                                                                           |
| <code>data/components/powerups.py</code>                                                                        | mushroom, 1-up, fire flower, star, fireball                                                                                                                                                                                               |
| <code>data/components/collider.py</code>                                                                        | invisible solid rectangles: ground, pipes, steps                                                                                                                                                                                          |
| <code>data/components/checkpoint.py</code>                                                                      | invisible triggers that release enemies and mark events                                                                                                                                                                                   |
| <code>data/components/flagpole.py</code> , <code>data/components/castle_flag.py</code>                          | the flagpole at the end and the castle flag                                                                                                                                                                                               |
| <code>data/components/info.py</code>                                                                            | the score bar: score, coins, world, time                                                                                                                                                                                                  |
| <code>data/components/score.py</code>                                                                           | the floating score numbers                                                                                                                                                                                                                |
| <code>resources/graphics/level_1.png</code>                                                                     | the level background                                                                                                                                                                                                                      |

| File (in each folder)                                 | What it is and what it holds                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>resources/graphics/mario_bros.png</code>        | Mario's sprites                                                                                                                                                                                                                                                                                                                  |
| <code>resources/graphics/smb_enemies_sheet.png</code> | enemy sprites                                                                                                                                                                                                                                                                                                                    |
| <code>resources/graphics/tile_set.png</code>          | bricks, boxes and tiles                                                                                                                                                                                                                                                                                                          |
| <code>resources/graphics/item_objects.png</code>      | coins, power-ups, flag                                                                                                                                                                                                                                                                                                           |
| <code>resources/graphics/text_images.png</code>       | the score bar's font                                                                                                                                                                                                                                                                                                             |
| <code>resources/music/</code>                         | 8 tracks: <code>main_theme</code> , <code>main_theme_sped_up</code> , <code>invincible</code> , <code>game_over</code> (.ogg); <code>death</code> , <code>flagpole</code> , <code>out_of_time</code> , <code>stage_clear</code> (.wav)                                                                                           |
| <code>resources/sound/</code>                         | 13 effects (.ogg): <code>big_jump</code> , <code>small_jump</code> , <code>brick_smash</code> , <code>bump</code> , <code>coin</code> , <code>count_down</code> , <code>fireball</code> , <code>kick</code> , <code>one_up</code> , <code>pipe</code> , <code>powerup</code> , <code>powerup_appears</code> , <code>stomp</code> |

The images and sounds are Nintendo's (see `docs/THIRD_PARTY_NOTICES.md`). No `__pycache__/` is written for the game: the project writes no bytecode caches (`common/__init__.py`).

## Brains, measured data and training evidence (mostly not in git)

Mostly too large for git; the important ones are hash-pinned in `artifacts.json` and checked by `tools/verify_artifacts.py`. Only the final brain's four files are in git: `FINAL_OBJECTIVE2.json` and `reachable_mask.npz` below, and the two root files above. `tools/final_brain.py` can also install them from the GitHub Release.

| File                                                                        | What it is and what it holds                                                     | Where it is used                            |
|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------|
| <code>checkpoints_qa/final_objective2_16000000/FINAL_OBJECTIVE2.json</code> | Objective 2's closure record: the approved 16M brain's hash and results (in git) | the dashboard approves the brain against it |
| <code>.../evidence/train.log</code>                                         | the final training run's log                                                     | evidence of how the 16M brain was made      |
| <code>.../evidence/console.log</code>                                       | the final training run's console output                                          | evidence                                    |
| <code>.../evidence/reward_telemetry.jsonl</code>                            | every reward channel, per episode, for the whole run (17 MB)                     | evidence; reward analysis                   |
| <code>.../evidence/coverage_audit_trail.jsonl</code>                        | coverage at every checkpoint of the run                                          | evidence; coverage history                  |
| <code>.../evidence/glitch_hunter_qa_16000000_steps_d906d09e.json</code>     | the 16M brain's 500-episode completion result (56.6%)                            | evidence                                    |

| File                                                                                                          | What it is and what it holds                                                     | Where it is used                                     |
|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------|
| <code>.../evidence/completion_baseline_6M.json</code>                                                         | the 6M baseline as it stood at closure                                           | evidence                                             |
| <code>checkpoints_qa/pre_main_6032768/glitch_hunter_qa.zip</code> , <code>..._coverage.npz</code>             | the 6.03M "healthy" checkpoint: the reference brain for anchor-KL                | loaded by anchor-KL training (the final run used it) |
| <code>checkpoints_qa/pre_main_6032768/reward_telemetry.jsonl</code> , <code>coverage_audit_trail.jsonl</code> | that checkpoint's own telemetry and coverage trail                               | record only                                          |
| <code>exploration_data/reachable_mask.npz</code>                                                              | the reachable-pixel mask: the denominator of every coverage % (in git)           | coverage, QA reward, tools                           |
| <code>exploration_data/jump_arcs.npz</code>                                                                   | the engine's real jump arcs (228 arcs)                                           | building the mask                                    |
| <code>exploration_data/observed_reach.npz</code>                                                              | places real play reached that the arc model had to keep                          | building the mask                                    |
| <code>exploration_data/coverage_bootstrap_6000000_mask_v4.npz</code>                                          | the 6M brain's routes, seeding the QA coverage map                               | QA training start                                    |
| <code>exploration_data/anchor_states.npz</code>                                                               | 6,007 frames the healthy brain sees, where anchor-KL is measured                 | anchor-KL training                                   |
| <code>evaluation/results/glitch_hunter_qa_16000000_steps_d906d09e.json</code>                                 | the 16M brain's completion verdict                                               | read when choosing what to resume                    |
| <code>generated/coverage_audits/glitch_hunter_main_brain_coverage_remaining_map.png</code>                    | the 16M brain's coverage map: red = reachable but never visited (84.25% covered) | reading only                                         |
| <code>generated/coverage_audits/glitch_hunter_main_brain_coverage_remaining_regions.json</code>               | the uncovered regions of that map, with sizes and positions                      | reading only                                         |

## Runtime output: generated/ (not in git)

Everything the project writes as it runs goes into `generated/`. The dashboard writes `generated/incidents/` and `generated/run_reports/` while it tests; they stay on this machine and are never overwritten; a report goes only when you delete it on the Bug History page. Right now `generated/incidents/` holds 8 real bugs found on the bugged game (all six bug types) and `generated/run_reports/` holds 2 clean runs. The tools



write `generated/coverage_audits/` and `generated/calibration_runs/`, training writes `generated/logs/`, and ruff, mypy, pytest and coverage keep their caches in `generated/cache/`.

| File                                                                              | What it is and what it holds                                                                                       | Where it is used                                                                                 |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>generated/incidents/INC-<br/>&lt;date&gt;-&lt;time&gt;-&lt;id&gt;/</code>   | one folder per bug found, read-only once written                                                                   | Bug Tracker, Bug History, <code>/incidents/...</code> downloads, <code>tools/incidents.py</code> |
| <code>.../incident.json</code>                                                    | the full record: detector, measurements, position, game and brain identity                                         | reports, Bug Tracker                                                                             |
| <code>.../trigger.png</code>                                                      | the exact frame the bug was detected, full resolution                                                              | reports, "Trigger frame" link                                                                    |
| <code>.../context_frames.zip</code>                                               | the stream frames just before the trigger                                                                          | the GIF, the PDF film strip                                                                      |
| <code>.../trajectory.json</code>                                                  | Mario's state and the action taken, frame by frame, before the trigger                                             | the reports; replay                                                                              |
| <code>.../context.gif</code>                                                      | the moments before the bug, then the trigger frame held                                                            | "GIF" link                                                                                       |
| <code>.../report.md</code> , <code>.../report.pdf</code>                          | the bug report in Markdown and PDF                                                                                 | "Markdown / PDF report" links                                                                    |
| <code>.../reproduction.json</code>                                                | the replay's verdict: reproduced or not                                                                            | reports, Bug Tracker                                                                             |
| <code>.../manifest.json</code>                                                    | SHA-256 of every file, and how each report was produced                                                            | <code>tools/incidents.py verify</code>                                                           |
| <code>generated/incidents/occurrences.jsonl</code>                                | every later sighting of an already-known bug (raises its "Seen" count)                                             | Bug Tracker counts                                                                               |
| <code>generated/incidents/_validation/</code>                                     | appears only after <code>tools/validate_incident_pipeline.py</code> ; its own test bugs, hidden from the dashboard | the validation tool                                                                              |
| <code>generated/run_reports/RUN-<br/>&lt;date&gt;-&lt;time&gt;-&lt;id&gt;/</code> | one folder per clean-game run that reached the castle                                                              | the Level complete card, Bug Tracker, Bug History                                                |
| <code>.../run.json</code>                                                         | the run's record: outcome, length, furthest point, score, bugs (none)                                              | the reports                                                                                      |
| <code>.../final.png</code>                                                        | the last frame of the run, full resolution                                                                         | "Final frame" link                                                                               |
| <code>.../finish.gif</code>                                                       | the run's last seconds                                                                                             | "GIF" link                                                                                       |
| <code>.../report.md</code> , <code>.../report.pdf</code>                          | the "No Bugs Found" report in Markdown and PDF                                                                     | report links, "Download all (.zip)"                                                              |

## Environment and caches (not in git)

All rebuilt automatically or by the install steps; described as folders because they hold thousands of generated files.

| Folder or file                                                                   | What it is and what it holds                                                                                                                                                                                                  | Where it is used                              |
|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| <code>venv_gpu/</code>                                                           | the project's Python 3.12.10 environment, about 26,500 files and 4.5 GB                                                                                                                                                       | every command, <code>run_dashboard.bat</code> |
| <code>venv_gpu/Lib/site-packages/</code>                                         | the 58 installed packages: PyTorch 2.14.0 + CUDA 12.6 (4.0 GB of it), Stable-Baselines3, pygame, Flask, NumPy, OpenCV, fpdf2, pytest, ruff, mypy and their dependencies                                                       | imported by all project code                  |
| <code>venv_gpu/Scripts/</code>                                                   | <code>python.exe</code> , <code>pythonw.exe</code> (no console, used for background helpers), <code>pip</code> , <code>pytest</code> , <code>ruff</code> , <code>mypy</code> , <code>pre-commit</code> and activation scripts | every command                                 |
| <code>venv_gpu/pyenv.config</code> , <code>Include/</code> , <code>share/</code> | the environment's settings (which base Python) and package extras                                                                                                                                                             | Python itself                                 |
| <code>.pytest_cache/</code>                                                      | pytest's cache (last failures); appears when pytest runs without <code>-p no:cacheprovider</code>                                                                                                                             | <code>pytest</code>                           |
| <code>.git/</code>                                                               | the full version history and GitHub link (about 80 MB)                                                                                                                                                                        | git                                           |

## Notes

- **torch is pinned to 2.14.0** (CUDA 12.6 build): it has no published security advisories, and it meets Stable-Baselines3's own torch requirement, so SB3 installs normally. It replaced 2.5.1 (CUDA 12.1) on 2026-09-25; both brains play move-for-move the same on it. Install torch first (so Windows gets the GPU build), then `requirements.txt` — or simply `uv sync --active`.
- **eventlet must never be installed.** Flask-SocketIO picks its async mode from what is installed; the dashboard runs in plain threading mode on purpose.
- **Headless everywhere except the dashboard:** training, evaluation, tests and incident replays use the SDL dummy video driver; only the dashboard opens a real game window.
- **Artifacts are protected by hash:** most brains, masks and coverage files are large and git-ignored (only the final 16M brain's four files are in git); `artifacts.json` (tracked) holds their SHA-256 values.